IOC Enrichment & Threat Intelligence Agent

A lightweight agentic CLI that lets a SOC analyst submit one or more IOCs (IPs, domains, URLs, file hashes — all four types) and get back a **grounded, source-cited enrichment verdict** from the abuse.ch feeds (ThreatFox, URLhaus, MalwareBazaar) — fast enough, and traceable enough, to hand to an incident commander in the next five minutes.

Built for the Deloitte Cyber Engineering pre-interview assignment (*IOC Enrichment & Threat Intelligence Agent*), time-boxed to ~1 hour of build.

```
┌──────────────────────── 185.220.101.45  [IPv4 address]
└──────────────────────────────────────────────────────┐

│ Verdict : MALICIOUS - confidence HIGH (ThreatFox reporter confidence
75/100;  │

│           corroborated by 2 independent feeds)
│

│ Class   : Botnet command & control (C2)
│

│ Malware : Cobalt Strike  (aliases: Agentemis, BEACON, CobaltStrike)
│

│ Seen    : first 2025-10-19 04:12 UTC   last 2026-05-18 11:02 UTC
[ACTIVE]    │

│ Action  : P1 - Act now - block at proxy/firewall and hunt for related
│

│           traffic; indicator is online or active within the last 90 days
│
```

```
| Evidence:
|
|   - ThreatFox IOC #1128873: 185.220.101.45:443, threat_type botnet_cc,
|
|     malware Cobalt Strike, confidence 75/100 ->
threatfox.abuse.ch/ioc/...    |
|   - URLhaus host: 2 malicious URL(s) recorded ->
urlhaus.abuse.ch/host/...    |
| Sources : ThreatFox: hit | URLhaus: hit   (queried 2026-07-20 20:15 UTC)
|
└──────────────────────────────────────────────────────────────────────────
──────┘
```

Quickstart

```
cd Assignment

python -m pip install -r requirements.txt      # requests + rich (+
anthropic, optional)


# 10-second offline demo - zero setup, zero network, always runs:

python -m ioc_agent --demo --file sample_iocs.txt


python -m ioc_agent "Is 185.220.101.45 a known malicious IP? What is it
associated with?"
```

```
python -m ioc_agent --file sample_iocs.txt      # batch + overall threat picture

python -m ioc_agent --stats 7                   # most active families, last 7 days

python -m ioc_agent                             # interactive analyst session

python -m ioc_agent --json "8.8.8.8"            # machine-readable audit output


# Self-checks:

python tests/test_iocs.py
```

The abuse.ch auth key (one real-world wrinkle)

The brief says the feeds "require no authentication" — **that changed in mid-2025**: abuse.ch now requires a free `Auth-Key` for all API queries (verified live during development: unauthenticated calls return `HTTP 401 {"error": "Unauthorized"}`). Getting one takes ~2 minutes:

1. Register (free) at **https://auth.abuse.ch/** and generate an auth key.
2. Provide it via any of: `ABUSECH_AUTH_KEY` env var, `--auth-key KEY`, or a local `abusech_auth.key` file next to this README.

Without a key the tool still runs and fails **honestly**: each source reports an actionable auth error and the verdict is `ERROR — treat as unknown, not clean`. And `--demo` mode always works — canned fixtures shaped exactly like real API replies, clearly banner-labelled — so the walkthrough can never be killed by conference-room Wi-Fi.

Optional: Claude analyst brief (`--llm`)

With `ANTHROPIC_API_KEY` set (and `pip install anthropic`), `--llm` appends a short Claude-written incident-commander brief **generated strictly from the already-synthesised evidence JSON** — the prompt forbids adding facts and requires per-claim source citations. Without a key the flag degrades to a one-line notice. The LLM never decides a verdict (see "Trust" below).

Requirements from the brief → where they're met

Requirement	Where
Accept IOC inputs (≥2 of the 4 types)	All **four** types handled: IP, domain, URL, MD5/SHA1/SHA256 hashes (`iocs.py`), incl. defanged forms (`hxxp://`, `185.220.101[.]45`)
Return enriched context (type, classification, family, confidence, first/last seen, source feed in every answer)	Every card: verdict + classification + family/aliases + confidence w/ rationale + seen-window + per-record source citations with links (`verdict.py`, `report.py`)
Handle unknowns honestly	`UNKNOWN — no records in the queried feeds` verdict with per-source "no records as of " lines + explicit caveat that absence ≠ clean; source outages produce `ERROR`, never a guess; invalid input is flagged separately and never queried (`verdict.py`, `cli.py`)
Agentic approach: query parsing → feed lookup → result synthesis	parse → plan → act → synthesise → report loop (`cli.py` docstring maps it); intent routing sends "most active families…" to stats, "summarise…" to the session summary
Bonus: batch + overall threat picture	`--file`, parallel lookups, summary table + threat-picture panel with P1/P2 call-outs and next steps

Something that runs	5 passing self-tests incl. an end-to-end pipeline check of all 7 sample IOCs; `--demo` guarantees it runs anywhere

Every example query in the brief works verbatim — paste it as the argument (or into the interactive prompt).

Architecture: the agentic loop

```
analyst free text ──► PARSE    extract + classify IOCs (refang defanged); detect
        │                          intents ("most active families", "summarise")
        ▼
      PLAN     per-IOC lookup plan by type          ip/domain → ThreatFox + URLhaus(host)
        │                                                 url       → URLhaus(url) + ThreatFox
        ▼                                                 hash      → MalwareBazaar + ThreatFox + URLhaus(payload)
      ACT      execute feed calls (parallel across IOCs, bounded workers,
        │      timeouts; a failed source is recorded, never fatal)
        ▼
  SYNTHESISE  deterministic rules fold evidence → verdict + confidence + priority, every bullet citing feed + record ID + link
        ▼
```

```
    REPORT    analyst cards • batch summary table • overall threat
picture

                • optional Claude brief (grounded on the same evidence)
```

Module	Responsibility
`ioc_agent/iocs.py`	Extraction/classification (strict regexes + `ipaddress` validation, refanging, appearance-order, dedupe)
`ioc_agent/feeds.py`	ThreatFox / URLhaus / MalwareBazaar clients; auth; timeouts; never-raise error model; demo-fixture switch
`ioc_agent/enrich.py`	Per-IOC lookup plan + bounded-parallel batch execution
`ioc_agent/verdict.py`	Deterministic synthesis: verdict, confidence ladder, priority, EICAR special-case, honest unknowns
`ioc_agent/report.py`	Rich/plain rendering, batch threat picture, JSON audit output, LLM evidence digest
`ioc_agent/llm.py`	Optional Claude narration (Anthropic SDK, `claude-opus-4-8`), strictly grounded, fails soft
`ioc_agent/cli.py`	Arg parsing, intent routing, interactive session (with session memory for "summarise")
`ioc_agent/demo_fixtures.py`	Offline API-shaped fixtures for the sample IOC list

How verdicts are derived (deterministic, explainable)

Verdict	When	Priority
`MALICIOUS`	Any feed returns records for the IOC	**P1** if URL online or last activity ≤ 90 days; else **P2** (block + retro-hunt)
`TEST FILE (EICAR)`	Hash/tags/AV names identify the EICAR AV test file	**P3** informational — reporting it "malicious" would be a textbook false positive
`UNKNOWN` (no records)	Every queried feed answered and none matched	**P4** — "no evidence in these feeds as of ", explicitly *not* "clean"
`ERROR` (inconclusive)	No hits **and** ≥1 source errored (network/auth/timeout)	**P3** — treat as unknown, re-run; names the failed source
`INVALID` input	Token looks like an attempted IOC but fails validation (bad IPv4 octets, wrong-length hash, unparseable line in a batch file)	Skipped **before** any lookup, listed in notices and the batch summary — never conflated with UNKNOWN

Confidence ladder: ThreatFox's own reporter-confidence score when present (≥75 high / ≥50 medium / else low) → else exact MalwareBazaar sample match (high) → else URLhaus URL online (high) / historical (medium); **corroboration by a second independent feed bumps one level**. The rationale is printed next to the number, so an analyst can always answer "why does it say high?".

Known-infrastructure context notes (8.8.8.8 = Google DNS, microsoft.com, …) are printed as *"local analyst note, not a feed verdict"* — they aid recognition but never alter a verdict.

Trust & traceability (why an analyst can defend this to an IC)

- **Verdicts are a pure function of feed records.** No model in the loop for judgement. Same inputs → same verdict, every time.
- **Every claim is cited**: feed name, record ID, and a clickable threatfox/urlhaus/bazaar.abuse.ch link the analyst can open to verify.
- **Every card shows what was checked**: `Sources: ThreatFox: hit | URLhaus: no records (queried <UTC time>)` — including what *couldn't* be checked.
- **`--json` is an audit record**: verdict + the raw, unmodified API responses that produced it. Pipe to a file and attach to the ticket.
- **The optional LLM only narrates.** It receives the already-decided evidence digest, is instructed to add nothing and cite per-claim, and if it's unavailable the deterministic summary stands. Trust does not depend on it.
- **Demo mode is labelled** with a banner on every run — canned data can never masquerade as live intel.

Design decisions & tradeoffs (1-hour box)

- **CLI over UI** — closest to SOC muscle memory, trivially scriptable (`--json` into a SOAR), and no build/deploy risk for "it must run".
- **All four IOC types** — the brief allows two, but ThreatFox covers every type through one endpoint, so full coverage cost little extra.
- **Deterministic core, optional LLM shell** — the fastest way to make an analyst trust an "AI" tool is to make the AI unable to invent verdicts.
- **Never-raise feed clients** — one dead feed degrades the answer, not the triage session; partial evidence is labelled as such.
- **Bounded parallelism (4 workers), 15s timeouts** — an 80-IOC incident batch finishes in ~1–2 min without hammering free community APIs.
- **Graceful-degradation ladder** — no `rich` → plain ASCII; no `requests` → demo still runs; no auth key → honest INCONCLUSIVE + how to fix; no `ANTHROPIC_API_KEY` → deterministic summary. Nothing takes the demo down.

- **Shortcuts taken, knowingly**: no response caching, no retry/backoff, no IPv6/email/CVE IOC types, no allowlist feed (top-1M) integration, minimal test surface, single-file audit via `--json` rather than a run store.

Reducing false-positive risk (discussion)

1. **No inference on absence** — unknowns are reported as "no records", never scored or guessed at. The feeds only contain *reported-malicious* indicators, and the output says exactly that.
2. **EICAR special-case** — the canonical trap for naive lookups: it appears in MalwareBazaar, but the tool labels it a harmless test artifact instead of a detection.
3. **Confidence is sourced, not invented** — ThreatFox's own reporter confidence is surfaced with its number; corroboration across independent feeds, and current online status, are the only boosters.
4. **Recency drives urgency** — a 2019 offline malware URL is P2 "investigate/retro-hunt", not a P1 fire alarm (see the tsunami.arm7 sample).
5. **Context notes for famous infrastructure** reduce the classic "analyst blocks 8.8.8.8" class of mistake — without whitelisting anything out of the verdict logic.
6. **Human stays in the loop** — output is decision support with links to primary sources; it recommends blocks, it doesn't execute them.

What it's good for today: first-five-minutes triage of an IOC batch during an active incident; consistent, citable answers instead of N analysts × N browser tabs; a threat picture an IC can act on.
What it is not: a detection system, a replacement for multi-source corroboration (VT, AbuseIPDB, passive DNS, internal telemetry), coverage of nation-state/low-prevalence indicators (abuse.ch skews crimeware), or an auto-blocker.

What I'd improve next

Retry/backoff + response caching (abuse.ch rate limits); AbuseIPDB and a top-1M allowlist as additional corroboration lanes; IPv6/email/CVE types; STIX 2.1 / MISP export and a SOAR webhook; a persistent per-incident case file (dedupe across re-runs, verdict diffing over time); richer LLM mode (natural- language follow-up questions over the evidence store); container packaging.

5-minute walkthrough script (interview deliverable #2)

1. **Problem framing (30s).** 80+ IOCs per incident, copy-paste triage, three analysts three answers. Bar: an answer the IC can trust in 5 minutes.
2. **Run it (90s).** `python -m ioc_agent --demo --file sample_iocs.txt` — walk one malicious card top to bottom: verdict → confidence *with reason* → family → seen-window → P1 action → cited evidence → sources line. Then the batch table + threat picture: "this last panel is the IC handoff."
3. **Honest unknowns (45s).** Point at 8.8.8.8 and the synthetic hash: "no records as of now" + caveat + context note; and EICAR: a lookup tool that *doesn't* cry wolf on the AV test file.
4. **Agentic loop (60s).** parse → plan → act → synthesise → report in `cli.py`; intent routing (`"most active families...?"` → stats mode) — `python -m ioc_agent --demo "What malware families have been most active in ThreatFox in the last 7 days?"`.
5. **Trust story (45s).** Deterministic verdicts, per-claim citations, `--json` raw-response audit trail; LLM narrates but never judges (show `--llm` if a key is handy).
6. **Judgment under pressure (30s).** The brief said "no auth" — live probing showed abuse.ch now 401s without a free Auth-Key (mid-2025 change), so the tool ships key handling, honest ERROR degradation, and an offline demo mode. Close with the shortcut list + next steps above.

(Live-mode fallback: if the room has no network or no key, every command in this script works with `--demo` — and says so on screen.)

Personal Project Guide - IOC Enrichment & Threat Intelligence Agent

This file is for your own understanding and interview preparation. It explains the entire assignment, the thought process behind the implementation, how the codebase is connected, why the current approach is strong, and exactly how to run the project smoothly in front of an interviewer.

The main public-facing README is `README.md`. This guide is more detailed and more candid.

---

1. What This Project Does

This project is a command-line IOC enrichment and threat intelligence agent.

An IOC, or Indicator of Compromise, is a technical artifact that may appear in security telemetry. Examples include:

- An IP address such as `185.220.101.45`
- A domain such as `emotet-c2-update.xyz`
- A URL such as `http://194.165.16.11/bins/tsunami.arm7`

- A file hash such as `44d88612fea8a8f36de82e1278abb02f`

The tool accepts analyst-style input, extracts IOCs from that input, queries threat intelligence feeds, and returns a grounded verdict for each IOC.

The key output is not just "bad" or "not bad". It gives:

- The IOC type
- A verdict
- Confidence and why that confidence was chosen
- Threat classification
- Malware family, if known
- First-seen and last-seen dates
- Recommended response priority
- Source-cited evidence
- Caveats for unknowns or failed lookups
- A batch-level overall threat picture
- Optional JSON output for audit trails
- Optional LLM narration, without letting the LLM decide verdicts

The project is intentionally built like something a SOC analyst could use in the first five minutes of an incident.

---

2. Assignment Goal In Plain English

The Deloitte-style cyber engineering prompt is asking for an agent that can:

1. Accept IOCs from a user.
2. Look them up in threat intelligence sources.
3. Enrich them with useful context.
4. Return an explainable verdict.
5. Handle unknowns honestly.
6. Show an agentic flow rather than a single static lookup.

This implementation goes beyond the minimum because it supports all four major IOC types:

- IPv4 addresses
- Domains
- URLs
- MD5/SHA1/SHA256 hashes

It also supports batch mode, demo mode, stats mode, JSON output, invalid input handling, and deterministic safety checks.

---

3. Thought Process Behind The Approach

The main design thought was:

A threat intelligence assistant is only useful if an analyst can trust and defend every verdict.

That led to several important decisions.

3.1 CLI Instead Of Web UI

A CLI is the fastest, most reliable way to deliver the assignment.

Why this was a good choice:

- SOC analysts are comfortable with terminals.
- It is easy to demo live.
- It is easy to script.
- It avoids frontend build complexity.
- It works well for batch IOC lists.
- It can output JSON for automation or ticket evidence.

For a one-hour or short assignment, this is a strong engineering tradeoff: focus on intelligence quality and trust rather than UI polish.

3.2 Deterministic Verdicts Instead Of AI Guessing

The tool has an optional LLM layer, but the LLM never decides whether an IOC is malicious.

The actual verdict is produced by deterministic code in `verdict.py`.

This is a very strong point:

- Same input plus same evidence gives the same verdict.
- No hallucinated malware family.
- No invented confidence score.
- Every evidence line points to a feed or record.
- The LLM can summarize only after the verdict has already been decided.

In an interview, say:

I deliberately kept the LLM out of the decision path. For a security tool, trust matters more than fluent wording. The model can narrate the evidence, but deterministic rules decide the verdict.

3.3 Honest Unknowns

The tool never says an IOC is "safe" simply because it was not found.

Instead, it says:

`UNKNOWN - no records in the queried feeds`

And it adds the caveat:

`Absence of evidence is not proof of benign.`

This is important because threat feeds are incomplete. An IOC may be new, private, targeted, or simply not reported to abuse.ch yet.

This is one of the best security-quality decisions in the project.

3.4 Separate ERROR From UNKNOWN

If the feed could not be checked because of auth, network, timeout, or bad API response, the tool returns:

`ERROR - one or more sources could not be checked`

It does not pretend that "no response" means "no records".

This matters because a broken source is not evidence of cleanliness.

In interview language:

`I separated no-hit results from failed lookups. A clean miss is UNKNOWN, but an API failure is ERROR and should be re-run.`

3.5 Demo Mode

The abuse.ch APIs may require an Auth-Key and network access. That could ruin a live interview demo.

So the project includes `--demo`.

Demo mode uses canned fixtures shaped like real abuse.ch API responses. This means the entire pipeline still runs:

parse -> plan -> feed result -> verdict -> report

Only the HTTP call is replaced.

This is another strong scoring point:

- The project always runs.
- The demo is deterministic.
- The demo clearly says it is canned data.
- The actual code path below the feed layer is still realistic.

3.6 Batch Summary For Incident Commander Use

The tool does not stop at individual IOC cards. For multiple IOCs, it also creates an overall threat picture:

- How many malicious
- How many unknown
- Which are P1
- Which are stale but still malicious
- Which malware families appear
- What the next steps should be

This makes the tool useful for more than lookup. It becomes a triage assistant.

---

4. High-Level Architecture

The whole project follows this agentic loop:

```
User input
   |
   v
Parse IOCs
   |
   v
Plan feed lookups
   |
   v
Query feeds or demo fixtures
   |
   v
Synthesize deterministic verdict
   |
   v
Render report, summary, JSON, or optional LLM brief
```

Each module owns one clean responsibility.

```
ioc_agent/
  __main__.py          Allows `python -m ioc_agent`
  __init__.py          Package metadata and version
  cli.py               User entry point and outer agent loop
  iocs.py              IOC extraction, refanging, classification, invalid
detection
  feeds.py             ThreatFox, URLhaus, MalwareBazaar clients and demo
mode
  enrich.py            Lookup planning and batch orchestration
  verdict.py           Deterministic verdict synthesis
  report.py            Terminal, JSON, summary, and stats rendering
  llm.py               Optional Claude summary layer
  demo_fixtures.py     Offline fixture data

tests/
  test_iocs.py         Parser and end-to-end demo pipeline tests
  test_verdict.py      Safety tests for verdict behavior
```

---

5. Codebase Walkthrough

5.1 `__main__.py`

This file makes the package runnable with:

```
python -m ioc_agent
```

It imports `main()` from `cli.py` and exits with the returned status code.

Why it matters:

- It gives a clean professional CLI entry point.
- It avoids needing users to run individual Python files directly.

5.2 `__init__.py`

This contains the package description and version:

```
__version__ = "0.1.0"
```

That version is used by:

```
python -m ioc_agent --version
```

5.3 `cli.py`

This is the outer control layer.

It handles:

- Argument parsing
- `--file`
- `--demo`
- `--json`
- `--stats`
- `--llm`
- `--auth-key`
- `--plain`
- Interactive mode
- Invalid input notices
- Calling enrichment
- Calling report rendering

Important functions:

- `parse_args()`: defines the CLI interface.
- `resolve_auth_key()`: checks command arg, environment variables, or local `abusech_auth.key`.
- `run_text()`: extracts IOCs from text, handles invalids, enriches valid IOCs.
- `run_stats()`: routes "most active families" requests to ThreatFox stats.
- `interactive()`: lets the analyst type multiple questions in one session.
- `main()`: decides which mode to run.

The agentic behavior is visible here:

```
parse input -> detect intent -> extract IOCs -> enrich -> report
```

Strong scoring point:

The CLI accepts both structured input and natural analyst questions, so it is more flexible than a rigid one-IOC-only script.

5.4 `iocs.py`

This module extracts and classifies indicators.

It supports:

- URLs
- Hashes
- IPv4 addresses
- Domains
- Defanged indicators
- Invalid candidate detection
- Deduplication
- Appearance-order preservation

Examples:

`hxxp://194.165.16[.]11/bins/tsunami.arm7`

becomes:

`http://194.165.16.11/bins/tsunami.arm7`

Important functions:

- `refang(text)`: converts defanged IOCs back into normal form.
- `classify(token)`: classifies a single token.
- `extract_with_invalid(text)`: returns valid IOCs and invalid-looking tokens.
- `extract_iocs(text)`: returns only valid IOCs.

Important implementation detail:

URLs are extracted before IPs and domains. This prevents the IP inside a URL from being counted as a separate IOC.

Strong scoring point:

The parser avoids false positives and separates invalid-looking indicators from true unknown feed results.

5.5 `feeds.py`

This module talks to the external threat intelligence feeds.

Feeds covered:

- ThreatFox
- URLhaus
- MalwareBazaar

It returns a `FeedResult` object for every lookup.

The key design rule:

Feed clients never raise errors into the rest of the app.

Instead, each feed result records:

- Source name
- Query made
- Whether the call was OK
- Whether records were found
- Raw records
- Error message, if any
- Query timestamp
- Raw API response

This lets `verdict.py` reason honestly about partial failures.

Demo mode is also implemented here:

```
enable_demo(FIXTURES)
```

When demo mode is enabled, feed functions return fixture data instead of making network requests.

Strong scoring point:

The feed layer is fault-tolerant and audit-friendly. It preserves raw responses for JSON output and never lets one bad feed crash the entire batch.

5.6 `enrich.py`

This module connects IOCs to feed lookups.

Important function:

```
plan_for(ioc)
```

Lookup plan by IOC type:

```
IP/domain -> ThreatFox + URLhaus host lookup
URL       -> URLhaus URL lookup + ThreatFox
MD5       -> MalwareBazaar + ThreatFox + URLhaus payload
SHA256    -> MalwareBazaar + ThreatFox + URLhaus payload
SHA1      -> MalwareBazaar + ThreatFox
```

Then:

```
enrich_one(ioc)
```

runs the plan and calls `synthesize()` from `verdict.py`.

For multiple IOCs:

```
enrich_many(iocs, max_workers=4)
```

uses a bounded thread pool.

Strong scoring point:

The code has a real plan-act-reason structure. It is not just hardcoded output.

5.7 `verdict.py`

This is the most important module.

It turns raw feed results into a single deterministic verdict.

Verdict levels:

```
malicious      -> confirmed hit in one or more feeds
test-file      -> EICAR test file, not a real threat
no-records     -> all queried feeds answered, none had records
inconclusive   -> one or more feeds failed and no positive hit resolved it
```

User-facing labels:

```
MALICIOUS
TEST FILE (EICAR) - not a real threat
UNKNOWN - no records in the queried feeds
ERROR - one or more sources could not be checked
```

Priority levels:

```
P1 -> Act now
P2 -> Investigate; malicious but stale/offline
P3 -> Informational or inconclusive
P4 -> No action from these feeds, but not clean
```

The confidence logic is also deterministic:

- ThreatFox reporter confidence is used when available.
- MalwareBazaar exact hash match is high confidence.
- URLhaus online URL is high confidence.
- Historical/offline URLhaus URL is medium confidence.
- Corroboration by multiple independent feeds bumps confidence.

The EICAR special case is excellent:

```
44d88612fea8a8f36de82e1278abb02f
```

This is the EICAR antivirus test file. Some naive tools may call it malware because it appears in malware datasets. This project correctly labels it as:

```
TEST FILE (EICAR) - not a real threat
```

Strong scoring point:

The verdict system is explainable, deterministic, and careful about false positives.

5.8 `report.py`

This module renders the output.

It supports:

- Rich terminal cards
- Plain ASCII output with `--plain`
- JSON output with `--json`
- Batch summary tables
- Overall threat picture
- Stats output
- Demo mode banner
- Optional LLM brief panel

The report card includes:

- Verdict
- Confidence

- Classification
- Malware family
- Seen dates
- Action
- Evidence
- Context note
- Caveats
- Source status line

Strong scoring point:

The report is built for analyst handoff, not just developer debugging.

5.9 `llm.py`

This optional module creates a short Claude-written incident commander brief.

Important security design:

- The LLM receives only the already-synthesized evidence JSON.
- The prompt says not to add facts.
- The LLM cannot change the verdict.
- If the key, SDK, or API is unavailable, it fails softly.

This is a good "AI agent" enhancement without making the core unsafe.

Strong scoring point:

The LLM is used for communication, not judgment.

5.10 `demo_fixtures.py`

This file contains canned abuse.ch-like responses for offline demo mode.

It includes sample data for:

- Cobalt Strike IP
- Emotet domain
- Historical Tsunami malware URL
- EICAR hash
- ThreatFox recent malware family stats

Unknown indicators fall back to realistic "no results" response shapes.

Strong scoring point:

Demo mode tests the real downstream logic while avoiding live network risk.

---

6. How The Modules Connect

Here is the flow for:

```
python -m ioc_agent --demo --file sample_iocs.txt
```

Step by step:

1. Python runs `ioc_agent/__main__.py`.
2. `__main__.py` calls `cli.main()`.
3. `cli.main()` parses arguments.
4. `--demo` enables fixture mode in `feeds.py`.
5. `--file sample_iocs.txt` is read.
6. `cli.run_text()` sends the file text to `iocs.extract_with_invalid()`.
7. `iocs.py` refangs and classifies each IOC.
8. `cli.py` calls `enrich.enrich_many()`.
9. `enrich.py` creates a feed lookup plan for each IOC.
10. `feeds.py` returns ThreatFox/URLhaus/MalwareBazaar-style results.
11. `verdict.py` synthesizes one verdict per IOC.
12. `report.py` prints cards, a summary table, and threat picture.

This is a clean separation of concerns.

---

7. Verdict Categories With Examples

7.1 MALICIOUS, P1

Example:

```
python -m ioc_agent --demo --plain "185.220.101.45"
```

Expected:

```
Verdict : MALICIOUS
Confidence: HIGH
Priority: P1
Family: Cobalt Strike
Sources: ThreatFox hit, URLhaus hit
```

Why P1:

- It is known malicious.
- It is active or recently seen.
- Multiple feeds corroborate it.

7.2 MALICIOUS, P2

Example:

```
python -m ioc_agent --demo --plain
"hxxp://194.165.16[.]11/bins/tsunami.arm7"
```

Expected:

```
Verdict : MALICIOUS
Confidence: MEDIUM
Priority: P2
Family: Tsunami
```

Why P2:

- It is confirmed malicious.
- But it is historical/offline/stale.
- It still deserves blocking and retro-hunting, but not immediate P1 panic.

7.3 TEST FILE, P3

Example:

```
python -m ioc_agent --demo --plain "44d88612fea8a8f36de82e1278abb02f"
```

Expected:

```
Verdict : TEST FILE (EICAR) - not a real threat
Priority: P3
```

Why this is strong:

- The tool avoids a false positive.
- It understands EICAR is an antivirus test artifact.

7.4 UNKNOWN, P4

Example:

```
python -m ioc_agent --demo --plain "8.8.8.8"
```

Expected:

```
Verdict : UNKNOWN - no records in the queried feeds
Priority: P4
Context: Google Public DNS resolver
Caveat: not clean, only no records in these feeds
```

Why this is correct:

- Threat feeds did not return records.
- That does not prove it is safe.
- Known infrastructure context helps the analyst avoid overreacting.

7.5 ERROR, P3

Run live mode without an abuse.ch key:

```
python -m ioc_agent --plain --timeout 5 "8.8.8.8"
```

Expected:

```
Verdict : ERROR - one or more sources could not be checked
Action  : P3 - Treat as UNKNOWN, not clean
```

Why this is correct:

- The feed failed due to missing auth.
- The tool does not treat failure as no records.

7.6 INVALID Input

Example:

```
python -m ioc_agent --plain "check 999.1.1.1 and hash
a3d5a6d5e4e2b8f1c9c4e2b8f1c9d4e"
```

Expected:

```
INVALID input skipped
No valid IOCs to look up
```

Why this is correct:

- `999.1.1.1` has an invalid IPv4 octet.
- The hash-like string has the wrong length.
- These are not queried and are not mixed with UNKNOWN.

---

8. How To Run The Project From Scratch

Open PowerShell in the project folder:

```
cd E:\Projects\Random\Resume\COMPANY\Deloitte-Cyber\Assignment
```

Install dependencies:

```
python -m pip install -r requirements.txt
```

Run tests:

```
python -m pytest -q
```

Expected:

```
14 passed
```

Run the offline demo:

```
python -m ioc_agent --demo --plain --file sample_iocs.txt
```

Run JSON audit output:

```
python -m ioc_agent --demo --json --file sample_iocs.txt
```

Run stats:

```
python -m ioc_agent --demo --plain --stats 7
```

Run a natural language query:

```
python -m ioc_agent --demo --plain "Is 185.220.101.45 a known malicious IP? What is it associated with?"
```

Run interactive mode:

```
python -m ioc_agent --demo --plain
```

Then type:

```
185.220.101.45
summarise
stats 7
help
quit
```

---

9. Perfect Interview Demo Script

Use demo mode unless the interviewer specifically asks for live API calls. Demo mode is reliable and avoids network/auth problems.

9.1 Start With One Sentence

Say:

I built a CLI-based IOC enrichment agent for SOC triage. It accepts IPs, domains, URLs, and hashes, queries abuse.ch-style threat feeds, and produces deterministic, source-cited verdicts with priorities and an overall incident summary.

9.2 Show Tests First

Run:

```
python -m pytest -q
```

Say:

The tests cover parsing, invalid handling, unknowns, errors, confidence, source failures, EICAR false-positive handling, and batch summary counts.

Expected:

```
14 passed
```

9.3 Run The Main Demo

Run:

```
python -m ioc_agent --demo --plain --file sample_iocs.txt
```

As output appears, point out:

- The banner says demo mode is canned.
- The malicious IP has two feed hits.
- Confidence explains why it is high.
- Evidence includes source links.
- The unknowns are not called safe.
- EICAR is handled as a test file.
- The stale URL is P2, not P1.
- The final threat picture is the incident handoff.

9.4 Explain The Best Card

Use `185.220.101.45`.

Say:

This card shows the main trust model: the verdict is malicious, confidence is high because ThreatFox has 75/100 reporter confidence and URLhaus corroborates it, the family is Cobalt Strike, and the evidence lines include source-specific references.

9.5 Explain Unknown Handling

Use `8.8.8.8`.

Say:

This is intentionally not labeled clean. It says no records were found in the queried feeds and adds a caveat that absence of evidence is not proof of benign. It also adds a local analyst note that this is Google Public DNS, but that context does not change the feed verdict.

9.6 Explain False Positive Reduction

Use the EICAR hash.

Say:

The EICAR test file appears in malware datasets, but it is not malware. I added a deterministic special case so the tool does not generate a false positive for a known test artifact.

9.7 Show JSON Output

Run:

```
python -m ioc_agent --demo --json --file sample_iocs.txt
```

Say:

JSON mode keeps the raw feed-shaped responses and the synthesized verdicts, so it can be attached to a ticket or consumed by SOAR-style automation.

9.8 Show Stats Intent

Run:

```
python -m ioc_agent --demo --plain --stats 7
```

Say:

I also added a simple analyst intent for active malware families. In live mode this would query recent ThreatFox IOCs, while demo mode uses realistic fixture data.

9.9 Explain Live Mode

Say:

abuse.ch now requires a free Auth-Key for live API access, so live mode supports `--auth-key`, `ABUSECH_AUTH_KEY`, or a local `abusech_auth.key` file. Without a key, the tool fails honestly with ERROR and says to treat the IOC as unknown, not clean.

If you have a key:

```
$env:ABUSECH_AUTH_KEY = "YOUR_KEY"
python -m ioc_agent --plain "8.8.8.8"
```

If you do not have a key, do not attempt live mode in the interview unless asked. Demo mode is enough and is safer.

---

10. Best Points To Score Highly

These are the project strengths you should emphasize.

10.1 Supports All Four IOC Types

The brief likely required at least two. This supports:

- IP
- Domain
- URL
- Hash

That is broader than the minimum.

10.2 Agentic Flow Is Clear

The tool does not just do a single lookup.

It follows:

```
parse -> plan -> act -> reason -> report
```

That maps directly to an agentic workflow.

10.3 Deterministic Security Logic

The verdict is not guessed by an LLM.

This is the most important trust argument.

10.4 Evidence Is Source-Cited

Every meaningful claim comes from feed records.

The report includes:

- Source name
- Record ID when available
- Links
- Query status
- Timestamps

10.5 Unknowns Are Honest

No records does not mean clean.

This shows security maturity.

10.6 Errors Are Honest

Failed feed calls become ERROR, not UNKNOWN and not SAFE.

This is a very important safety property.

10.7 False Positive Awareness

EICAR handling is a thoughtful detail.

It shows you understand that threat intelligence is not just matching strings.

10.8 Batch Threat Picture

The final summary gives an incident commander useful next steps.

This makes the tool operationally useful.

10.9 Demo Mode Is Reliable

The assignment is easy to run in any room.

You can say:

I built demo mode because threat intelligence demos should not depend on conference Wi-Fi or external API auth.

10.10 JSON Output

This makes the output auditable and automation-friendly.

10.11 Tests Cover Safety Properties

The tests are not only happy-path tests. They check:

- No records is unknown, not safe.
- API errors are errors, not unknown.
- Partial failures are disclosed.
- Confidence comes from source data.
- Corroboration raises confidence.
- Malformed API response is an error.
- Invalid input is tracked separately.

This is excellent for scoring.

---

11. Current Test Results

On this machine, the following passed:

```
python tests\test_iocs.py
```

Result:

```
all 6 tests passed
```

```
python tests\test_verdict.py
```

Result:

```
all 8 tests passed
```

```
python -m pytest -q
```

Result:

```
14 passed
```

Also checked:

```
python -m compileall -q ioc_agent tests
python -m ioc_agent --version
```

Version output:

```
ioc-agent 0.1.0
```

---

12. What To Say If Asked About Tradeoffs

Good answer:

I optimized for trust, traceability, and something that runs reliably. I chose a CLI because it is scriptable and close to SOC workflows. I used deterministic verdict rules because security judgments should be explainable. I added demo fixtures because live threat-feed demos are fragile. Given more time, I would add caching, retries, more feeds like AbuseIPDB or VirusTotal, STIX/MISP export, and persistent incident case storage.

---

13. Known Limitations

Mention these confidently if asked:

- No caching yet.
- No retry/backoff yet.
- No IPv6 support.
- No email/CVE/file-path IOC support.
- Live mode needs abuse.ch auth.
- abuse.ch is crimeware-heavy and not complete coverage.
- No persistent case database.
- No SOAR webhook yet.
- LLM mode is optional and depends on an API key.

These are reasonable limitations for an assignment prototype.

---

14. Best Final Summary For Interview

Use this as your closing explanation:

The main idea was to build a threat intelligence assistant an analyst could actually defend. It extracts IOCs from messy input, chooses the right feed lookups by IOC type, handles network and auth failures honestly, synthesizes deterministic verdicts with source-cited evidence, and produces both per-IOC cards and a batch-level threat picture. The LLM layer is optional and only narrates evidence; it never decides the verdict. That keeps the project useful as an AI-assisted tool without sacrificing trust.

---

15. Quick Command Cheat Sheet

Run all tests:

```
python -m pytest -q
```

Best full demo:

```
python -m ioc_agent --demo --plain --file sample_iocs.txt
```

Single malicious IP:

```
python -m ioc_agent --demo --plain "185.220.101.45"
```

Defanged URL:

```
python -m ioc_agent --demo --plain
"hxxp://194.165.16[.]11/bins/tsunami.arm7"
```

EICAR hash:

```
python -m ioc_agent --demo --plain "44d88612fea8a8f36de82e1278abb02f"
```

Unknown IOC:

```
python -m ioc_agent --demo --plain "8.8.8.8"
```

Invalid input:

```
python -m ioc_agent --plain "999.1.1.1"
```

Stats:

```
python -m ioc_agent --demo --plain --stats 7
```

JSON:

```
python -m ioc_agent --demo --json --file sample_iocs.txt
```

Interactive:

```
python -m ioc_agent --demo --plain
```

Version:

```
python -m ioc_agent --version
```